

Plain2Metta in Detail

Purpose, Architecture, Evidence Pipeline, Worked Examples, and Next-Phase Roadmap

Expanded technical report for Benjamin Goertzel

Revised 19 August 2026

Abstract

Plain2Metta turns a constrained Plain specification into a chain of inspectable, immutable artifacts, generated MeTTa and Python, bounded runtime observations, and obligation-specific validation evidence. Its central idea is not that natural-language intent can be proved automatically. It is that every interpretive and executable step can be made explicit, independently reviewable, hash-bound to its inputs, and invalidated when an upstream byte changes. This report explains the implemented system for a reader who knows the basic ideas of Plain, MeTTa, and Lean 4 but has not followed Plain2Metta’s development. It covers the two historical layers, the complete artifact lineage, Stages 1–11 of semantic validation, the G0–G6 evidence vocabulary, the Hypothesis/TLC/Z3/Lean/Hyperon/Python back-ends, the API and browser path, and three exact worked examples. It also explains the conceptual purpose of the larger Plain2Metta pursuit, summarizes lessons from the prototype, gives design guidance for four substantially more realistic benchmark tasks, and compares a single-agent work plan with a six-agent parallel plan. It separates demonstrated behavior from broader research ambitions and records acceptance and VM2 deployment evidence.

Contents

1	Introduction: purpose, design rationale, and lessons so far	4
1.1	The larger purpose of Plain2Metta	4
1.2	Why the design is artifact-centered and fail-closed	4
1.3	What the prototype has taught us	5
2	Orientation: what problem Plain2Metta solves	6
2.1	What the current release does and does not claim	6
3	The two layers in the repository	7
3.1	The earlier source-preserving compiler	7
3.2	The v2 immutable project and semantic-validation layer	7
4	The ordinary v2 artifact chain	7
4.1	Review is attached to bytes, not filenames	8
4.2	Generated code is inert until a separate handoff	8

5	The general semantic-validation model	8
5.1	Independence and correlated error	9
6	Evidence is a vector: G0 through G6	9
7	Stages 1–11, step by step	10
7.1	Stage 1: preserve source and materialize semantic artifacts	10
7.2	Stage 2: finite contract calculus	10
7.3	Stage 3: independently authored plan, review, and approval	10
7.4	Stage 4: property and example checks with Hypothesis	11
7.5	Stage 5: finite-state checking with TLC	11
7.6	Stage 6: SMT checking with Z3	11
7.7	Stage 7: Lean 4 kernel checking	11
7.8	Runtime evidence: Hyperon MeTTa and bounded Python	12
7.9	Stage 8: conservative cross-tool composition	12
7.10	Stage 9: evidence projection for API and browser	12
7.11	Stage 10: calibrated release metadata	12
7.12	Stage 11: clean-checkout release acceptance	13
8	The HTTP request and browser path	13
9	Worked example 1: greeting with trace preservation	13
9.1	Exact Plain source	13
9.2	Generated/reference MeTTa and expected observation	14
9.3	Generated/reference Python and expected observation	14
10	Worked example 2: task validation and owner preservation	14
10.1	Exact Plain source	14
10.2	Generated/reference MeTTa and expected observations	15
10.3	Generated/reference Python and expected observations	15
11	Worked example 3: forecast split, horizon, and baseline	15
11.1	Exact Plain source	15
11.2	Generated/reference MeTTa and expected observations	16
11.3	Generated/reference Python and expected observations	16
12	Adversarial and fail-closed behavior	17
12.1	Source and approval attacks	17
12.2	Evidence substitution and framing attacks	17

12.3 Behavioral and backend failures	17
12.4 Browser and route integrity	17
13 Acceptance evidence and deployment	17
13.1 Accepted and merged revisions	17
13.2 VM2 deployment	18
14 Trust boundaries and security posture	18
15 What has actually been accomplished	18
16 Limitations and open research	19
16.1 Restricted semantic coverage	19
16.2 Interpretation remains the central epistemic risk	19
16.3 Evidence coverage and calibration	19
16.4 Providers, UI breadth, and execution infrastructure	20
16.5 Promising next investigations	20
17 How to read a Plain2Metta result responsibly	20
18 Reproducibility record	20
19 Conclusion	21
A Guidance for authoring the next realistic benchmark tasks	21
A.1 Rules that apply to every benchmark	21
A.2 Benchmark A: transactional reservation and payment saga	22
A.3 Benchmark B: concurrent authorization with revocation	23
A.4 Benchmark C: temporal ML evaluation and deployment decision	24
A.5 Benchmark D: evidence-based policy decision under conflict and ignorance	25
A.6 Portfolio-level acceptance	26
B Two work plans for the next development phase	26
B.1 Plan S: one coding agent	27
B.2 Plan P: six parallel coding agents	28
B.3 Coordination controls for the parallel plan	29

1 Introduction: purpose, design rationale, and lessons so far

1.1 The larger purpose of Plain2Metta

The long-term purpose of Plain2Metta is not merely to save the labor of typing MeTTa programs. It is to investigate whether human-scale statements of software intent can become the stable, inspectable center of a development process in which machines elaborate, formalize, implement, test, and revise software without losing the relationship between what was requested and what was built. Plain supplies a comparatively accessible language for stating concepts, requirements, assumptions, tests, and risks. MeTTa supplies a compositional symbolic substrate in which those objects and their relationships can be represented, queried, transformed, and eventually reasoned about. Grounded languages such as Python supply efficient ordinary computation; model checkers and solvers explore bounded formal consequences; Lean supplies kernel-checked proofs for the fragment precise enough to merit them. Plain2Metta is the controlled bridge among these roles.

That bridge is important for AI-assisted development. A capable model can produce plausible code from prose, but plausibility is not traceability. A second model can review the code, but agreement between models is not proof of shared meaning. Tests can pass while testing an accidental interpretation; a theorem can be true while formalizing the wrong proposition. If the original intent, intermediate interpretation, validation plan, generated program, and evidence are transient prompt context, it is difficult to locate where an error entered or to know what a later edit invalidates. The pursuit therefore asks a more demanding question than “can an LLM translate Plain to MeTTa?”: can a mixed human-machine process make each consequential semantic commitment explicit, reviewable, executable where appropriate, and retractable when its premises change?

A successful answer could support a form of continuously revisable software in which requirements are not dead prose beside the implementation. They would remain linked to executable examples, properties, formal obligations, runtime monitors, and generated components. Changes could trigger semantically targeted regeneration and revalidation rather than an undirected rewrite. MeTTa is attractive in this picture not only as a target language but as a medium for representing the evolving network of requirements, interpretations, implementations, evidence, conflicts, and uncertainty.

1.2 Why the design is artifact-centered and fail-closed

The prototype deliberately decomposes translation into immutable artifacts instead of treating a model response as the compiler. This follows from four conceptual observations.

First, meaning is version-relative. Approval of one set of source bytes does not approve a reworded document, even if a human considers the edit minor. Content-addressed identities and transitive invalidation make that dependency machine-checkable. They are not administrative overhead; they are the mechanism by which evidence remains about the object it purports to support.

Second, different tools establish different kinds of facts. Hyperon can execute MeTTa reductions; Python can perform grounded computation; Hypothesis can search generated cases and shrink counterexamples; TLC can exhaust a declared finite transition system; Z3 can establish satisfiability or unsatisfiability of an encoding; Lean can kernel-check a theorem. None of these automatically establishes that the encoding captures the author’s intent. Backend ownership and the G0–G6 vector preserve these distinctions instead of compressing heterogeneous evidence into a misleading confidence number.

Third, interpretation and validation are correlated failure sources. If the same generation step invents the meaning, writes the implementation, and chooses the tests, it can create a self-consistent but wrong world. The reviewed-contract and independently authored validation-plan boundaries do not guarantee

independence, but they make role separation possible and leave an auditable object at which a human or another model can disagree.

Fourth, honest incompleteness is valuable. Real specifications contain ambiguity, missing policy, environmental assumptions, and claims that cannot be settled by the available tools. An explicit typed hole or Unknown verdict is more useful than synthesized certainty because it directs the next question. Failed admission is therefore a semantic feature: unsupported text is prevented from inheriting evidence produced for a nearby supported profile.

1.3 What the prototype has taught us

The prototype has produced several practical lessons that should guide the next phase.

1. **Provenance is implementable, but exactness propagates.** Exact UTF-8 spans, canonical serialization, content hashes, immutable review targets, and downstream invalidation can be made to work end to end. Doing so revealed that seemingly small boundary choices—newline handling, duplicate keys, path normalization, current-version checks, and review revocation—are part of semantic correctness rather than peripheral hygiene.
2. **A narrow vertical is more informative than broad mock coverage.** Running three exact examples through the real API, artifact store, approved plan, formal/property tools, Hyperon and Python observations, server-side verdict composition, and browser exposed integration errors that component demonstrations hid. The next benchmarks should again be complete vertical slices, not isolated impressive backend examples.
3. **Evidence must be per obligation.** A program may satisfy output shape while losing trace identity, enforce a happy path while mishandling retries, or prevent leakage while failing to report uncertainty. One overall pass badge obscures these distinctions. Obligation-specific ancestry and grades must remain visible as examples become larger.
4. **Formal machinery is useful only when its claim boundary is shown.** The prototype successfully invokes Hypothesis, TLC, Z3, and Lean with pinned, attributed artifacts. It also demonstrates how easy it would be to overstate them. A bounded TLC check is not an unbounded proof; a Z3 result is about an encoding; a Lean theorem is about its exact statement and assumptions. The UI and reports should expose scope, formulas, theorem text, and non-applicability.
5. **Mutation and adversarial tests reveal evidence-pipeline bugs.** Source edits, stale plans, missing approvals, backend hash mismatches, timeouts, malformed evidence, runtime disagreement, and false properties exercised the trust boundaries better than more happy-path examples would have. Future benchmarks need semantic mutants designed before implementation so that validation is evaluated rather than merely demonstrated.
6. **The present bottleneck is semantic breadth, not infrastructure existence.** The artifact schemas, bounded adapters, composer, API vertical, and acceptance machinery now exist. The live system remains narrow because admission and profiles are curated around three exact texts and elementary semantic shapes. The next development phase should generalize only what realistic benchmarks force it to generalize.
7. **Parallel development requires frozen seams.** The historical 11-stage plan was appropriate for building trust boundaries in dependency order. Repeating it serially for every new case would waste the modularity it created. Once benchmark contracts and shared interfaces are frozen, case fixtures, lowerings, oracles, mutants, and documentation can proceed in parallel, with continuous integration guarding the common spine.

These lessons motivate the appendices. They do not yet supply finished benchmark specifications. Instead they define how those specifications should be authored and how their implementation can be organized without confusing concurrency with the removal of dependencies.

2 Orientation: what problem Plain2Metta solves

Plain is useful because a technical intent can be written in a form that is more disciplined than ordinary prose while remaining readable by a domain expert. MeTTa is useful because relationships, rules, and symbolic transformations can be represented and executed as atoms. Lean 4 is useful because a precisely stated proposition can be checked by a small proof kernel. The difficult part is the space *between* these representations. Nothing about translating an English sentence into a MeTTa rule guarantees that the rule captures what its author meant, and nothing about successfully executing a program makes its output semantically correct.

Plain2Metta addresses that gap as an evidence-management and compilation pipeline. It asks, in order:

1. What exact bytes did the author review?
2. Which concepts, requirements, tests, contracts, assumptions, and holes were extracted from those bytes?
3. Which interpretation was approved, and against which exact version?
4. What code was generated from that interpretation?
5. Under what runtime and resource bounds did that code execute?
6. Which independent examples, properties, models, solver obligations, or theorems support each requirement?
7. What remains unknown, conditional, or operationally unobserved?

This framing matters. A one-shot “Plain in, MeTTa out” demo tends to hide interpretation choices. Plain2Metta instead preserves those choices as artifacts. A reviewer can trace a verdict backward to evidence, from evidence to an approved validation plan and implementation, and from those objects to the exact source clauses that motivated them.

2.1 What the current release does and does not claim

The merged release implements a source-preserving, content-addressed, fail-closed pipeline and a real evaluation UI vertical for three exact, reviewed examples. It includes a finite typed contract calculus, strict semantic artifacts, bounded formal and runtime backends, server-side evidence composition, and adversarial invalidation tests.

Claim boundary: It is not a general natural-language program synthesizer and does not prove that arbitrary Plain prose has been understood. The three live examples use exact reviewed profiles. A reworded or unsupported document is not allowed to inherit their evidence. Passing examples are bounded evidence, not a universal proof. A Lean theorem proves only the theorem actually encoded, under the assumptions actually admitted. A successful deployment canary is an operational observation over that canary interval, not permanent production correctness.

3 The two layers in the repository

The public repository is named Plain2Metta; the internal package and intermediate representation retain the name `specatom_hs`. Two layers coexist because the project evolved from a conservative source compiler into a larger reviewed artifact workflow.

3.1 The earlier source-preserving compiler

The `plain_to_metta` layer indexes a Plain file without throwing away its origin. It records file digests, sections, bullet items, concepts, requirements, tests, and exact byte/line spans. Stable identities and provenance let a downstream atom point back to the characters that produced it. Conservative passes recognize explicit markers such as concepts, requirements, coverage labels, claims, assumptions, evidence, risks, mitigations, owners, deadlines, data-flow relations, temporal ordering, and selected ML/security methodology cues.

The emphasis is conservative extraction. Unsupported predicates, ambiguous coverage, missing evidence, placeholder owners, vague deadlines, unresolved references, or raw-text-only semantics become questions or refusals rather than confident invented atoms. The PeTTa/MeTTa backend emits only records admitted by its profile; malformed arity, broken source links, unsupported semantic levels, or unsafe scalar encodings fail closed.

This layer answers: “What structure can we reproducibly preserve and expose from the source?” It intentionally does not answer: “What executable program should arbitrary prose mean?”

3.2 The v2 immutable project and semantic-validation layer

The `specatom_hs` v2 layer turns translation into a staged project. It adds immutable artifact versions, SHA-256 content hashes, exact upstream references, hash-bound review decisions, logical-IR admission, generated output handoff, bounded test results, semantic artifacts, formal/runtime evidence, traceability, and transitive invalidation.

The two layers are complementary. The first preserves source structure and supplies conservative semantic scaffolding. The second controls what may be promoted from a reviewed source into an interpretation, implementation, test, or verdict. Existing schema-v1 artifacts remain readable; new semantic meaning is introduced through new versioned schemas rather than silently changing an old envelope.

4 The ordinary v2 artifact chain

Before discussing semantic-validation Stages 1–11, it helps to understand the underlying project chain. A typical project contains the following ordered artifacts:

Artifact	Purpose
Original specification	Exact submitted Plain bytes and their hash.
Elaborated specification	A proposed clarification/expansion bound to the original.
Test specification	Proposed tests and acceptance conditions, independently reviewable.
Reviewed snapshots	Exact approved elaborated and test bytes; comments and decisions bind to versions.
Logical IR	Typed declarations, contracts, obligations, dependencies, provenance, and explicit holes; no executable body.
Logical review	Machine-readable findings, reviewer dispositions, rationale, and compile-blocking state.
Compiler request/log	Exact reviewed inputs, guidance, provider attribution, response schema, and request digest.
Compiler output	Inert generated files with safe paths, hashes, sizes, and requirement traceability.
Sandbox handoff	Exact admitted generated bundle and bounded adapter request.
Test result	Per-test observation, status, output, duration, covered requirement IDs, adapter identity, and request digest.
Traceability report	Reconstruction from source through tests, classifying each requirement as passing, failing, skipped, or untested.

Each arrow is explicit. An artifact reference contains an identity, version, kind, and content hash. Deserialization recomputes hashes and rejects forged or malformed provenance. Generic artifact creation cannot bypass the special admission gates for logical IR, compiler output, or results.

4.1 Review is attached to bytes, not filenames

A mutable filename such as `requirements.plain` is not a trustworthy review target: its contents can change after approval. Plain2Metta attaches annotations and decisions to an immutable artifact reference. The Phase-3 review compares exact elaborated and test snapshots and records reviewer, timestamp, target, decision, and rationale. Logical findings likewise bind to the exact current logical IR. Repairing or waiving a critical finding creates a new review version; it does not rewrite history.

Compilation is admitted only if the current reviewed inputs match the current IR, the exact IR is approved, and no critical finding remains open or deferred. If the source, elaboration, tests, approval, or IR changes, descendants become stale. This is *transitive invalidation*: a verdict cannot remain current when its semantic ancestry no longer exists.

4.2 Generated code is inert until a separate handoff

The compilation boundary stores generated text but marks it `executed:false`. It neither publishes files nor imports Python nor evaluates MeTTa. Paths must be safe, fields strict, and each generated file must carry requirement traceability. Execution requires a separate sandbox handoff and an explicitly injected out-of-process adapter. The coordinator makes one call, supplies no provider selection or retry policy, and rechecks the current ancestry after the call before admitting a result.

This separation limits authority. A compiler may propose bytes; it does not gain ambient authority to execute them. An evidence runner may execute a specific admitted request; it cannot reinterpret the source, approve a plan, or mutate the project.

5 The general semantic-validation model

The semantic vertical extends the artifact spine with four key concepts.

SemanticContract A typed, finite statement of inputs, output, preconditions, postconditions, invariants, effects, temporal constraints, assumptions, and unresolved holes, with source-clause ancestry.

ValidationObligation A particular claim linked to a contract and source clauses, with required evidence grade, admissible methods, finite domain, assumptions, severity, and unresolved state.

ValidationPlan Independently authored examples, generators, properties, state models, differential oracles, and formal tasks, approved against exact contract and source hashes.

RuntimeEvidence and ValidationVerdict Tool-attributed observations and a conservative composition for one exact obligation, implementation, plan, and set of evidence artifacts.

The contract language is deliberately finite and typed. Its core supports literals, variables, state reads, Boolean and numeric operators, bounded quantification, trace predicates, explicit UTC time, approximation, and approved assumption references. Unknown operations, free prose, free variables, malformed types, unapproved assumptions, and typed holes block execution. A hole is an honest record of unresolved meaning, not executable placeholder code.

5.1 Independence and correlated error

The design separates interpretation author, implementation author, validation author, and evidence runner. The validation author may inspect approved source and contracts but should not derive expected answers from generated implementation bodies. Otherwise the same misunderstanding can occur in both implementation and oracle. Full independence is sometimes impossible; when dependencies are shared, they are named and the evidence claim is limited.

The exact-profile evaluation fixtures are a useful example of this caution. They provide deterministic and inspectable G3 oracles, but their generator and expected outputs are curated in a shared profile data file. That supports the three reviewed examples; it is not evidence of general semantic induction.

6 Evidence is a vector: G0 through G6

Plain2Metta does not collapse unlike evidence into a single confidence number. Each obligation receives a Boolean vector with exactly G0–G6 plus status, assumptions, holes, counterexamples, attribution, and residual risk.

Grade		Meaning and common misinterpretation
G0	pre-served	Exact reviewed source bytes, identities, hashes, and ancestry are known. This proves provenance, not meaning.
G1	structured	Contract and validation-plan representations pass schema, typing, provenance, and consistency checks. A well-formed interpretation may still be the wrong interpretation.
G2	executable	An admitted artifact starts and terminates under its declared bounded runtime. Exit zero alone says nothing about expected behavior.
G3	examples	Independently specified concrete examples pass, including complete normalized output where required. Coverage is limited to those examples.
G4	properties	Property-based, metamorphic, state-machine, differential, or bounded model checks pass over a recorded domain, budget, and seed/model. This remains bounded evidence.

Grade	Meaning and common misinterpretation
G5 proved	A precisely stated formula is established by a named solver, exhaustive finite checker, or proof kernel under explicit assumptions and bounds. It is not automatically a proof of the original prose.
G6 observed	A deployed system satisfies monitored obligations over a recorded operational interval. Observation does not imply universal future behavior and is not stronger than proof in a simple linear sense.

The vector form prevents silent promotion. G2 does not imply G3; G4 does not imply G5; and G6 is operationally different from G5. A policy may require a particular combination, but the response retains the components. The legacy `semantically_validated` Boolean exists only for compatibility with the old three-profile UI and denotes that its exact dual-runtime G3 oracle passed.

7 Stages 1–11, step by step

Stages 1–8 execute per supported evaluation request. Stage 9 projects their stored evidence to the API. Stage 10 is immutable release calibration metadata validated at service construction, not rerun per request. Stage 11 is a clean-checkout release gate, not a runtime stage.

7.1 Stage 1: preserve source and materialize semantic artifacts

The service accepts only an exact bundled reviewed source. Stage 1 creates strict `plain2metta-semantic-artifact/v` envelopes for contracts and obligations, binding every identity and SHA-256 hash to ordered upstream references and source clauses. Canonical IDs cover complete documents. Storage validates the envelope against immutable project ancestry rather than trusting caller-supplied references.

Adversarial tests reject forged, partial, extended, duplicated, stale, path-confused, hash-mismatched, or unknown-version documents. A source-byte mutation invalidates the contract $\rightarrow$ obligation $\rightarrow$ plan $\rightarrow$ evidence $\rightarrow$ verdict chain. Stage 1 therefore establishes G0-style provenance and the substrate for G1; it does not interpret arbitrary prose correctly by itself.

7.2 Stage 2: finite contract calculus

Stage 2 validates and lowers the typed contract calculus, runs its deterministic reference interpreter, and can produce a canonical MeTTa projection. The interpreter defines what supported predicate forms mean, rather than delegating semantics to whichever backend happens to run them. Execution ownership is explicit: symbolic relations may belong to MeTTa; ground numeric or systems operations may belong to Python.

A typed hole, unsupported operation, invalid type, unapproved assumption, or unbounded construct blocks promotion. This is important because it prevents prose from being smuggled into a purportedly executable field. Stage 2’s success means “this approved finite representation has deterministic semantics,” not “the representation perfectly captures human intent.”

7.3 Stage 3: independently authored plan, review, and approval

Stage 3 constructs exactly one validation plan for the approved contracts and obligations. The plan names concrete examples, input generators, properties, models, solver tasks, proof tasks, finite budgets,

and coverage claims. It is reviewed as its own immutable artifact. Approval binds the exact plan, contracts, obligations, and source hashes.

Missing, rejected, revoked, edited, stale, or mismatched approval prevents execution. Validation authors are not given generated implementation bodies by default, reducing correlated-oracle error. A plan-byte change invalidates all results derived from it. This stage is where the human or independent author can reject a plausible but incorrect interpretation before expensive or misleading evidence is generated.

7.4 Stage 4: property and example checks with Hypothesis

Stage 4 executes only Hypothesis tasks named in the approved plan. The pinned identity is Hypothesis 6.138.15. A generator is itself reviewed: its domain, distribution, budget, shrinker, and seed belong in the evidence. A failure retains the minimized counterexample. A pass reports bounded cases; it is not rephrased as proof over an infinite domain.

The evaluation contract bounds this adapter to 3 seconds wall time, 2 CPU seconds, 128 MiB memory, a 1 MiB file limit, and one process. Malformed or misattributed output, a request-hash mismatch, timeout, or failing property is conservative evidence, never a pass inferred from process exit alone.

7.5 Stage 5: finite-state checking with TLC

Stage 5 runs an approved finite-state model when the obligation has an applicable temporal/state interpretation. The pinned checker is TLC 1.7.4; the recorded `tla2tools.jar` SHA-256 is `936a262061c914694dfd669a543be2`. It runs under a project-local Temurin JRE 21.0.12+8.

The adapter is bounded to 8 seconds wall time, 5 CPU seconds, 3 GiB memory, an 8 MiB file limit, and one worker. A counterexample trace is preserved. If TLC is genuinely inapplicable, the result is a justified `Unknown`, not an invented pass and not an error disguised as non-applicability.

7.6 Stage 6: SMT checking with Z3

Stage 6 checks an approved SMT obligation with pinned Z3 4.15.3. It can establish satisfiability/unsatisfiability or preserve a model/counterexample for the exact encoded formula. Bounds are 5 seconds wall time, 4 CPU seconds, 512 MiB memory, and a 4 MiB file limit. The evidence schema identifies the tool, request digest, formula ancestry, observations, and interpretation.

Z3 evidence is not relabelled as a Lean theorem. A solver result remains dependent on the encoding and solver identity. Unknown, timeout, malformed evidence, or disagreement with another runtime prevents unsupported grade promotion.

7.7 Stage 7: Lean 4 kernel checking

Stage 7 checks an approved theorem using Lean 4.33.0 and Mathlib v4.33.0 at commit `db584cd6d46c92f209a44c0f1c8`. The recorded Lean archive SHA-256 is `4b3fb03c29a1e0a253fb1d11f9bae3725f19a0dc6fc09b3ea16d2c9df3349e2c`. It is bounded to 45 seconds wall time, 30 CPU seconds, 8 GiB memory, a 64 MiB file limit, and one thread.

The important distinction is between kernel checking and prose validation. The kernel can establish the encoded proposition under imports and assumptions. Plain2Metta retains the source/contract/theorem ancestry so a reviewer can inspect whether that proposition addresses the intended obligation. Failed compilation, timeout, a theorem that does not close, or a hash mismatch remains failure/unknown

evidence. TLC or Z3 output cannot be presented as Lean evidence merely because it supports a similar claim.

7.8 Runtime evidence: Hyperon MeTTa and bounded Python

The Stage 4–7 orchestration also admits exact plan-authorized implementation observations. Generated MeTTa runs with pinned Hyperon CLI 0.2.10, under a 2-second timeout. Generated Python runs in an isolated subprocess with a 2-second timeout, 2 CPU seconds, 128 MiB memory, a 1 MiB file limit, and one process. Neither adapter accepts a client-selected executable, shell command, network capability, credential, or arbitrary host path.

The dual-runtime examples compare complete normalized stdout against reviewed expected output. Runtime success is G2; matching an independent example is G3. A zero exit code cannot override wrong output, and disagreement between MeTTa and Python is a failure rather than something averaged away.

7.9 Stage 8: conservative cross-tool composition

Stage 8 joins only admitted evidence to each exact obligation. It checks kind, attribution, request hash, tool identity, ancestry, applicability, observations, assumptions, and required method. The composer produces status (pass, fail, unknown, or blocked), the exact G0–G6 vector, counterexamples, holes, assumptions, and residual risk.

No single passing backend can fill missing required evidence. A property pass cannot stand in for a proof; a Lean proof cannot fabricate operational observation; an inapplicable TLC model cannot become a pass. Failures and unknowns remain visible. This stage owns the grade decision; neither the API route nor browser JavaScript may infer it.

7.10 Stage 9: evidence projection for API and browser

Stage 9 reloads the canonical chain and returns schema `plain2metta-evaluation-evidence/v1`. Its `ancestry.stages` reports 1 through 8. Every node includes stage, kind, artifact ID, content hash, state, and upstream references. Verdicts bind obligation, implementation, approved plan, evidence refs, and a complete Boolean G0–G6 vector. The projection also supplies plan/review/approval state, backend attribution, counterexamples, assumptions, holes, residual risk, and Stage-10 metadata.

Artifact bodies are not dumped into the default response. They remain behind exact-ID, exact-hash, authorized download boundaries. The browser renders the server objects. An independent audit found that it reads `grade_achieved.vector`; the earlier client-side grade inference and hard-coded G4–G6 behavior are absent.

7.11 Stage 10: calibrated release metadata

Stage 10 is not recomputed for every request. A strict checked-in data artifact records release `plain2metta-r02-stage` corpus schema/hash, acceptance examples, categorized mutations, and calibration result. Service construction validates the bytes once. Missing, malformed, changed, or unbound metadata makes the service unavailable rather than selecting convenient defaults.

The fixed relevant mutation corpus scored 8/8 killed. One cosmetic-wording mutation was classified as non-semantic. The score is evidence about this finite corpus and policy; it is not a probability that the system is correct.

7.12 Stage 11: clean-checkout release acceptance

Stage 11 is an external acceptance gate. Its script exercises the real `/api/evaluate` route, Stage 1–9 traversal, Stage-10 consumption, pinned backends, adversarial invalidation, browser framing, live examples, compilation, and repository hygiene. It also checks that no credential-like content, oversized changed file, whitespace error, or unexpected untracked artifact entered the candidate.

At accepted task head `1755dddc31dc02c04d7db35dec01ba1fb6b9215`, the focused real-route suite passed 106/106 and the complete provider-free suite passed 777/777. The fresh clone rebuilt 3,015 pinned Lean jobs and passed. A separate static audit confirmed that the Flask route calls the Stage 1–8 orchestrator, Stage 9 contains Stage-10 metadata, and the browser consumes server grades. These are release-acceptance observations, not proof of unrestricted Plain semantics.

8 The HTTP request and browser path

The public evaluation surface is deliberately narrow. `POST /api/evaluate` accepts exactly a duplicate-free JSON object with one field:

```
{"text": "<UTF-8 Plain source>"}
```

Both request and decoded source are bounded to 128,000 bytes. The text must byte-match one of the three files returned by `GET /api/examples`. Blank, mutated, reworded, malformed, duplicate-key, oversized, or unsupported input fails closed without promoting partial state. The client cannot supply artifact IDs, plans, approvals, expected answers, backend choices, grades, credentials, executable paths, or download authority.

For accepted input, the server creates an isolated canonical project chain, executes the ordered vertical, reloads the stored evidence, and then runs the compatibility/reference evaluator. Repository failure, concurrent ancestry change, invalid evidence, or any trust-boundary error aborts atomically. There is no retry, fallback provider, network access, host-shell authority, or partial success committed as current.

The browser is therefore a presentation layer, not an evidence authority. It shows ancestry, reviews, tools, observations, counterexamples, assumptions, holes, risks, and the G vector returned by the server. This design makes a UI bug less capable of upgrading evidence: the canonical verdict remains on the server and in immutable artifacts.

9 Worked example 1: greeting with trace preservation

The first example is intentionally small. It demonstrates that even a pure transformation has two distinct requirements and that trace preservation is not implied merely by producing greeting text.

9.1 Exact Plain source

```
***definitions***
- :Greeting: is a text response associated with a request trace.

***requirements***
- [id:GREET-1] The service shall return a :Greeting: for each request.
- [id:GREET-2] The :Greeting: shall retain the request trace identifier.
```

The exact curated profile is `greeting-trace-v1` with shape `pure`. The IDs matter: assertions link separately to GREET-1 and GREET-2, so an output that returns “hello” but drops `trace-123` cannot discharge both.

9.2 Generated/reference MeTTa and expected observation

```
(= (make-greeting $text $trace) (Greeting $text $trace))
!(make-greeting "hello" "trace-123")
```

The reviewed expected Hyperon output is:

```
[(Greeting "hello" "trace-123")]
```

The rule exposes the semantic shape directly: the two inputs become two fields of a `Greeting` atom. The query supplies a concrete text and trace. Exact-output comparison detects missing trace, changed spelling, additional unexpected output, or a different constructor.

9.3 Generated/reference Python and expected observation

```
def make_greeting(text, trace):
    return {'text': text, 'trace': trace}
def main():
    value = make_greeting('hello', 'trace-123')
    print(f"Greeting(text={value['text']},trace={value['trace']})")
```

Expected stdout is exactly:

```
Greeting(text=hello,trace=trace-123)
```

The assertions are “response text equals hello” for GREET-1 and “response retains trace-123” for GREET-2. The Python representation is not treated as the definition of MeTTa semantics; it is a separately executed grounded reference shape. Agreement gives stronger concrete-example evidence than one runtime alone, while the shared curated profile remains a documented dependency.

10 Worked example 2: task validation and owner preservation

The second example introduces a conditional rejection path and a stateful domain shape. It separates three obligations: store valid data, reject invalid data, and preserve ownership.

10.1 Exact Plain source

```
***definitions***
- :Task: is a named unit of work owned by a :User:..

***requirements***
- [id:TASK-1] A :User: shall add a valid :Task: to the task list.
- [id:TASK-2] The service shall reject a :Task: without a name.
- [id:TASK-3] The service shall preserve the owner of every stored :Task:..
```

The profile `task-validation-owner-v1` has shape `stateful`. Its examples exercise both the accepted and rejected branch rather than deriving rejection behavior from the happy path.

10.2 Generated/reference MeTTa and expected observations

```
(= (add-task $name $owner)
  (if (== $name "")
      (Rejected "missing-name")
      (StoredTask $name $owner)))
!(add-task "Write report" "alice")
!(add-task "" "alice")
```

Expected output is two complete results:

```
[(StoredTask "Write report" "alice")]
[(Rejected "missing-name")]
```

The first result covers successful storage and shows that `alice` is preserved. The second is a negative boundary example. Merely terminating or returning some `Rejected` atom is not enough; the normalized output must contain the reviewed `missing-name` reason.

10.3 Generated/reference Python and expected observations

```
def add_task(name, owner):
    return ({'status':'rejected', 'reason':'missing-name'}
            if not name else
            {'status':'stored', 'name':name, 'owner':owner})
def main():
    valid = add_task('Write report', 'alice')
    invalid = add_task('', 'alice')
    print(f"stored={valid['name']} owner={valid['owner']}")
    print(f"rejected={invalid['reason']}")
```

Expected stdout is:

```
stored=Write report owner=alice
rejected=missing-name
```

The three linked assertions are: a valid named task is stored (TASK-1), a blank name is rejected (TASK-2), and stored owner equals `alice` (TASK-3). This example also illustrates why grades are per obligation. A program could pass TASK-1 and TASK-2 while accidentally replacing every owner with a default user; TASK-3 must remain independently visible.

11 Worked example 3: forecast split, horizon, and baseline

The third example addresses temporal lineage and an elementary anti-leakage constraint. It is not a forecasting benchmark. Its purpose is to make data split order, horizon, baseline identity, and reporting observable.

11.1 Exact Plain source

```
***definitions***
- :Observation: is a timestamped numeric measurement.
- :Forecast: is a prediction for a declared future horizon.
```

```

***requirements***
- [id:FORECAST-1] The pipeline shall train only on :Observation:
  records earlier than the evaluation window.
- [id:FORECAST-2] The pipeline shall emit one :Forecast: for the
  declared horizon.
- [id:FORECAST-3] The evaluation shall compare each :Forecast:
  with a named baseline.
- [id:FORECAST-4] The evaluation shall report the split timestamps
  used for each result.

```

The profile is forecast-split-horizon-baseline-v1, shape temporal-lineage.

11.2 Generated/reference MeTTa and expected observations

```

(= (eligible-train $time $start) (< $time $start))
!(eligible-train 10 20)
!(eligible-train 30 20)

```

Expected Hyperon output is:

```

[True]
[False]

```

This isolates the ordering predicate behind FORECAST-1: time 10 is eligible before evaluation start 20, while future time 30 is not. The MeTTa fixture does not pretend to train a model. It validates the symbolic split predicate, leaving numerical/reporting behavior to the grounded Python example.

11.3 Generated/reference Python and expected observations

```

def main():
    train = [x for x in [10, 19, 30] if x < 20]
    print(f'train={train} evaluation_start=20')
    print('forecast=42 horizon=24h')
    print('baseline=seasonal-naive delta=2')

```

Expected stdout is:

```

train=[10, 19] evaluation_start=20
forecast=42 horizon=24h
baseline=seasonal-naive delta=2

```

The assertions are: future timestamp 30 is excluded (FORECAST-1), one 24-hour forecast is emitted (FORECAST-2), the named `seasonal-naive` baseline is compared (FORECAST-3), and split timestamp 20 is reported (FORECAST-4). The constant forecast value 42 and delta 2 are fixture values, not a scientific claim. The useful result is that the critical lineage fields are explicit and exactly checked.

This example shows backend ownership clearly. MeTTa represents a temporal eligibility relation. Python grounds list filtering and formatted evaluation output. Both observations retain requirement links. A future, richer forecasting profile could add generators, uncertainty methods, real data identifiers, and formal leakage obligations without changing the provenance model.

12 Adversarial and fail-closed behavior

A trustworthy pipeline must be tested not only on its happy path but on ways evidence could be accidentally or maliciously upgraded. The accepted suite covers the following classes.

12.1 Source and approval attacks

Exact source mutation, unsupported rewording, changed contracts or obligations, changed plan bytes, stale immutable review, missing approval, rejected approval, and approval revocation all invalidate the appropriate descendants. Tests check proportional invalidation: upstream state is preserved, but evidence and verdicts that depended on it cease to be current. This prevents “approved once, valid forever” behavior.

12.2 Evidence substitution and framing attacks

The system rejects malformed, partial, duplicate-key, unknown-version, path-confused, hash-mismatched, or misattributed artifacts. Backend results must carry the exact request digest and tool identity. A TLC/Z3 observation cannot masquerade as Lean proof evidence. Artifact-body download requires the exact ID and hash plus authorization; metadata routes do not confer mutation or execution authority.

12.3 Behavioral and backend failures

Wrong expected output, cross-runtime disagreement, a failing property, failing TLC model, failing Lean theorem, backend timeout, malformed Stage-4 evidence, or resource exhaustion remains fail/unknown/blocked according to the approved policy. Process exit zero cannot override observations. An inapplicable formal method yields a justified unknown record. Missing required evidence prevents grade promotion even if another backend passes.

12.4 Browser and route integrity

Tests instrument the real Flask route rather than a detached service helper. They assert ordered Stage 1–8 traversal, Stage-9 projection, Stage-10 metadata, complete G0–G6 server vectors, duplicate-safe JSON, size bounds, exact source admission, and absence of client-side grade synthesis. This closes a common integration gap in which well-tested backend code is silently bypassed by the actual UI route.

13 Acceptance evidence and deployment

13.1 Accepted and merged revisions

The evaluation-UI task branch was accepted at 1755dddc31dc02c04d7db35dec01ba1fb6b9215. PR #3 was then merged to `main` as 5ce102cb02e72f377314205b28f102e5fc39911f. The merge did not alter the accepted head’s content.

Observed: The Stage-11 task-branch run passed 106/106 focused real-route tests in 53.975 seconds and 777/777 complete provider-free tests in 70.501 seconds, followed by byte compilation, diff, credential-pattern, large-file, and untracked-file checks. A fresh clone replay rebuilt 3,015 Lean jobs. A separate audit confirmed the intended server/browser code path.

These counts describe exact recorded environments and revisions. Earlier incremental runs include expected red gates and test-harness mistakes; they were preserved rather than rewritten as successes. Examples include a missing `PYTHONPATH`, a test that patched a read-only `PosixPath` method, and intermediate failures before the next stage existed. Corrected runs are separately recorded. This makes the development evidence auditable instead of presenting only a polished endpoint.

13.2 VM2 deployment

The merged commit is installed in an immutable release directory on the existing VM2. One enabled and active `plain2metta-vm2.service` worker is bound to `127.0.0.1:8081`. It is localhost-only, not publicly exposed. No provider resource was provisioned, started, resized, or retained for this deployment.

Observed: Corrected live canaries passed `GET /`, `health`, `examples`, and `POST /api/evaluate` for all three bundled sources. Each response had Stage 1–8 ancestry, Stage-9 schema `plain2metta-evaluation-evidence/v` Stage-10 release `plain2metta-r02-stage10-20260817`, and all server-derived G0–G6 keys.

Two prior deployment trials failed safely. First, root-owned Lean cache trace files were unreadable by the unprivileged service. Second, shell command substitution stripped the exact source’s trailing newline, so source admission correctly returned HTTP 422. Each trial rolled back before correction. The newline incident is especially instructive: exact byte identity is not merely documentation; it changed the route outcome as designed.

14 Trust boundaries and security posture

Trusted core operations are narrow: canonical serialization, hash calculation, strict schema validation, exact-chain reconstruction, and invalidation. Plain prose, model/provider output, generated code, validation plans, backend stdout/stderr, uploaded names, PR text, and runtime dependencies are untrusted inputs. Hyperon, Python, Hypothesis, TLC, Z3, Lean, Mathlib, and their launchers are identified dependencies, not magically part of the semantic trusted core.

The web client has no command authority. Runtime adapters are injected and command-free from the caller’s perspective. Network, credentials, arbitrary host paths, and shell commands remain outside the request schema. Bounded JSON and duplicate-key rejection reduce framing ambiguity. Atomic repository admission prevents half-written evidence chains. One-call/no-retry behavior limits unreviewed mutation and cost; provider selection and credential policy are deliberately not hidden in the core.

This does not make generated execution risk-free. Resource limits, process isolation, localhost binding, exact handoffs, and absence of ambient authority are defense layers. A production deployment would still need OS/container hardening, monitoring, patch management, and a deliberate exposure policy.

15 What has actually been accomplished

The work can be summarized as a progression from syntax preservation to evidence-bearing execution:

1. Exact Plain bytes, spans, stable identities, requirements, tests, concepts, semantic markers, questions, and conservative PeTTa atoms are preserved rather than flattened into opaque generated code.
2. Immutable project artifacts, SHA-256 provenance, strict persistence, version-bound comments/decisions, atomic writes, and transitive invalidation create a trustworthy state spine.

3. Reviewed elaboration/test snapshots and non-executable logical IR make interpretive decisions inspectable. Critical gaps block compilation unless explicitly repaired or waived with attribution and rationale.
4. Provider-neutral one-call elaboration/compilation seams store inert output and exact provenance without bundling credentials, retries, or ambient execution.
5. Separate sandbox handoff/results and traceability reconstruct how each requirement maps to generated paths and passing/failing/skipped/untested tests.
6. Strict semantic contracts, obligations, validation plans, evidence, counterexamples, and verdicts provide a finite general framework beyond the original compatibility Boolean.
7. Hypothesis, TLC, Z3, Lean, Hyperon, and Python are integrated as distinct, pinned, bounded evidence sources. Attribution and scope are retained rather than conflated.
8. The real evaluation API now traverses Stages 1–8, returns a Stage-9 projection with Stage-10 calibration, and the browser renders the server’s complete per-obligation G vector.
9. The adversarial matrix, clean-checkout Stage-11 gate, independent audit, merge, and localhost-only VM2 deployment have concrete recorded evidence.

16 Limitations and open research

16.1 Restricted semantic coverage

The largest limitation is semantic breadth. The finite calculus and explicit profiles cover reviewed forms; they do not understand unrestricted English or the full Plain language. The earlier compiler intentionally extracts only explicit markers and conservative patterns. Rich policies, domain-specific units, probabilistic claims, complex concurrency, open-world knowledge, and underspecified operational behavior need new typed forms and review.

The three examples are pedagogically useful but tiny. They demonstrate provenance, negative cases, temporal split lineage, and dual-runtime evidence; they do not demonstrate large programs, long-lived mutable state, distributed systems, realistic ML training, or autonomous synthesis.

16.2 Interpretation remains the central epistemic risk

Formal machinery can verify the wrong formalization perfectly. Hashes prove identity, not adequacy. Independent roles, source links, explicit holes, and human approval reduce risk but do not eliminate it. Future work should study how reviewers compare source clauses to contracts, how disagreements are represented, and how multiple independent interpretations can be evaluated without mistaking consensus for truth.

16.3 Evidence coverage and calibration

The 8/8 mutation result is based on a fixed categorized corpus. Larger and less profile-shaped mutation sets are needed, including semantic-preserving changes, subtle oracle corruption, backend attribution attacks, generator bias, proof/contract mismatch, and operational drift. Mutation score should remain a diagnostic, not be converted into a correctness probability.

G6 requires real monitored obligations over an explicit interval. A live canary is useful deployment evidence, but sustained operational observation, data retention, incident semantics, and privacy/security policies remain deployment-specific work.

16.4 Providers, UI breadth, and execution infrastructure

The provider-neutral seams do not bundle a production model provider, credential manager, retry/fallback policy, or cost policy. The full v2 product UI envisaged editors, workflow bars, logical-review drill-down, generated-code inspection, results, and version sidebars; the shipped evaluation UI is the narrow evidence vertical, not that full authoring environment.

The core deliberately lacks a general host/container executor. The live vertical uses explicitly configured bounded adapters. Broader execution will need a carefully reviewed sandbox and publication boundary. Public exposure of the VM2 service has not been authorized or implemented.

16.5 Promising next investigations

Useful next steps include: a richer library of typed contract patterns; larger independently authored examples; explicit unit/dimensional semantics; state-machine and temporal patterns with source-level explanations; proof obligation generation whose adequacy can itself be reviewed; independent validation-plan providers; differential execution across MeTTa runtimes; metamorphic relations for symbolic transformations; realistic ML leakage and uncertainty profiles; operational monitors that produce honest G6 records; and usability studies of whether reviewers can find planted interpretation errors from the evidence graph.

17 How to read a Plain2Metta result responsibly

For a practical evaluation, begin with the source and obligation, not the headline. Check that the bytes and clause IDs are the intended version. Read the contract, assumptions, and holes. Verify who or what authored and approved the validation plan. Inspect which backend produced each evidence item, its bounds, seed/model/theorem, and exact observation. Treat a counterexample as first-class. Confirm that the verdict's plan and implementation hashes are current. Finally, interpret each G bit according to its own meaning rather than imagining a scalar maturity score.

For the three bundled examples, the correct conclusion is specific: the exact reviewed sources traversed the current immutable pipeline; their curated MeTTa and Python fixtures produced exact expected observations under bounded runtimes; the approved formal/property tasks and server composer produced the recorded vectors; and the merged service replayed the path on VM2. The incorrect conclusion would be that Plain2Metta can now prove arbitrary Plain documents equivalent to arbitrary MeTTa programs.

18 Reproducibility record

The public repository is <https://github.com/bgoertzel-sing/plain2metta>. The accepted task head is 1755dddc31dc02c04d7db35dec01ba1fb6b9215; merged main is 5ce102cb02e72f377314205b28f102e5fc39911f. The main acceptance command is `scripts/run-stage11-acceptance.sh` from a clean checkout with the recorded project-local pinned dependencies and Lean prebuild. The detailed experiment records are retained in the project notebook under:

- `experiments/20260818T065419Z-plain2metta-evaluation-stage11-vertical-acceptance/;`
- `experiments/20260818T072517Z-plain2metta-evaluation-stage11-clean-checkout/;`
- `experiments/20260818T075514Z-plain2metta-evaluation-stage11-independent-audit/;` and
- `experiments/20260818T153337Z-plain2metta-pr3-merge-vm2-deployment/.`

The source of truth for exact examples is `examples/evaluation/*.plain`; generation/expected-output profiles are in `src/specatom_hs/evaluation_profiles.json`; immutable Stage-10 data is in `src/specatom_hs/vert` and the route contract is `docs/evaluation-vertical-contract.md`. These files outrank this expository narrative if a future revision changes behavior.

19 Conclusion

Plain2Metta’s substantive result is an explicit chain of accountability from Plain source to evidence. Exact bytes and hashes prevent unnoticed drift; reviewed contracts and plans expose interpretation; inert compilation and separate bounded execution constrain authority; multiple backends retain their distinct epistemic meanings; conservative composition prevents one success from filling unrelated gaps; and the API/browser show the resulting ancestry rather than a hand-waved badge.

The present system is deliberately narrow in semantic coverage but unusually explicit about what it knows. That is a sound foundation for broader Plain-to-MeTTa research: new contract forms, generators, proofs, and operational monitors can be added without abandoning provenance, role separation, exact review, or fail-closed behavior.

A Guidance for authoring the next realistic benchmark tasks

This appendix is a design brief for the person or coding agent who will write the benchmark specifications. It intentionally stops short of supplying the finished Plain documents, formal contracts, expected outputs, or complete fixture data. Those objects should be authored as separately reviewable artifacts. Prematurely filling them in here would make the implementation appear more settled than it is and would weaken the separation between benchmark intent and system construction.

A.1 Rules that apply to every benchmark

Each benchmark should describe a small but credible system rather than a large fictional product. It should be complicated enough that several requirements interact and a superficially plausible implementation can be wrong. Its environment must be finite or fixture-bounded so that another person can reproduce the result locally. External services, private data, and nondeterministic network dependencies should be represented by explicit interfaces and recorded traces rather than required for the acceptance run.

The benchmark author should create the following objects before a coding agent implements a new semantic profile:

1. a short domain narrative naming actors, state, boundaries, and the decision or service the software performs;
2. a Plain source document with stable requirement IDs, definitions, explicit assumptions, non-goals, and unresolved questions;

3. an obligation table stating what would count as Pass, Fail, Unknown, Blocked, and inapplicable for each requirement;
4. a proposed typed-contract representation, with unsupported meaning retained as typed holes rather than encoded by approximation without review;
5. a backend-ownership table: which claims are checked by reference interpretation, property tests, model checking, SMT, Lean, MeTTa, grounded execution, or operational observation;
6. deterministic fixtures and independent oracles, including exact expected observations where exactness is meaningful and tolerance or statistical procedures where it is not;
7. a preregistered mutant set containing realistic semantic errors, not only syntax corruption;
8. expected G0–G6 vectors per obligation, including justified zeros and acceptable Unknowns; and
9. an end-to-end acceptance recipe that reconstructs every result from exact source, tool versions, seeds, scopes, and artifact hashes.

At least one requirement in each benchmark should be deliberately outside the initial supported fragment. The desired outcome for that requirement is an honest question, typed hole, or Unknown verdict. This tests whether broader inputs cause the system to invent semantics. At least one mutant should be self-consistent across implementation and naive tests, so that it is caught only by an independently stated invariant or oracle.

A.2 Benchmark A: transactional reservation and payment saga

Why this case

This case should exercise state, idempotency, partial failure, compensation, and externally visible effects. A bounded reservation/payment workflow is realistic enough to expose errors common in distributed services while still admitting a finite model. It extends the toy idempotency examples into a system in which two resources must not silently diverge.

System envelope to specify

The benchmark author should choose a concrete small workflow, such as reserving one inventory item and authorizing one payment for a request bearing an idempotency key. Define the state machines for reservation, payment, and the coordinating request. State exactly which effects are durable, which messages may be duplicated, delayed, reordered, or lost, where a crash may occur, and what retry policy is assumed. Decide whether compensation means release, refund, cancellation request, or eventual manual review; these are not interchangeable.

Candidate obligations should include: no completed order without both valid effects; at most one captured payment per idempotency key; no oversold item; duplicate requests return a result consistent with the first committed outcome; recovery after each enumerated crash point; and an explicit terminal or escalated state rather than silent limbo. Liveness claims must state their fairness and availability assumptions. If the first benchmark only checks safety in a finite scope, progress should remain Unknown rather than being inferred from absence of a counterexample.

Expected representations and evidence

The Plain document should identify actors, commands, events, states, effects, and invariants with stable IDs. The logical IR should preserve the distinction between a requested, authorized, captured, released, and refunded effect. Its contract calculus should use bounded traces, explicit state transitions, idempotency-key equality, and a finite inventory count. MeTTa should represent the workflow relations and permit queries such as whether a trace contains a terminally consistent pair of effects. Grounded Python should execute a deterministic reference saga over scheduled event traces.

TLC is the natural owner of bounded interleavings, duplicate delivery, crash points, recovery, and safety invariants. Hypothesis should generate command and failure schedules and shrink an observed violation. Z3 can check local state/invariant consistency and boundary conditions. Lean should initially be reserved for a small, reviewed theorem—for example, that a particular pure transition preserves an at-most-once token invariant—rather than a claim about the whole distributed workflow. G6 should remain false unless a separate deployed monitor observes real transactions over a declared interval.

Mutants and questions the author must settle

Include mutants that: record an idempotency key after rather than before an effect; capture twice after a retry; decrement inventory twice; acknowledge success before durable commit; lose a compensation request; conflate payment authorization with capture; recover from stale state; or reuse a key across tenants. Require source-linked counterexample traces.

Before finalizing the benchmark, settle the meaning of “exactly once,” the atomicity available inside each service, the fate of permanently failing compensation, the scope of tenant identity, and whether eventual progress is part of the contract. These questions should not be decided implicitly by a generated model.

A.3 Benchmark B: concurrent authorization with revocation

Why this case

This benchmark should test policy, identity, temporal ordering, cache freshness, and race conditions. Authorization looks like a Boolean function in a toy example, yet real decisions depend on principal, resource, action, delegation, policy version, context, and revocation time. A stale allow is a useful example of a program that can pass ordinary unit tests while violating the intended security invariant.

System envelope to specify

Define a small multi-tenant document or workflow service. Include direct roles, one bounded form of delegation, policy-versioned authorization decisions, a cache, audit events, and revocation. Specify whether a request is judged at arrival, decision, or effect-commit time; how clock/order is modeled; what consistency the policy store provides; and the maximum permitted cache staleness, if any.

Candidate obligations should cover tenant isolation; default deny for missing or malformed identity; action/resource-specific permission; delegation scope; revocation behavior; cache invalidation; preservation of request trace and policy version in the audit record; and non-disclosure of sensitive policy details in a denial. Include a policy conflict whose desired result is explicitly unresolved until a precedence rule is approved.

Expected representations and evidence

The logical IR should distinguish facts, derived permissions, denials, delegations, policy versions, and effect events. MeTTa is well suited to representing and querying role/delegation relationships, provided negation and closed-world assumptions are explicit. Python can ground request processing, cache behavior, and exact audit output. Hypothesis should generate principals, resources, action combinations, policy changes, and short concurrent traces.

TLC should explore orderings among authorization, revocation, cache refresh, and effect commit. Z3 can check finite policy consistency, unreachable allow conditions, and tenant separation in the supported fragment. Lean may prove a small non-escalation or tenant-separation theorem for a precisely defined policy evaluator. The benchmark must show the exact theorem and its treatment of stale external state; it must not describe the theorem as proving the deployed authorization system secure.

Mutants and questions the author must settle

Mutants should include: omit tenant from a cache key; treat missing identity as anonymous read; allow role names to collide across tenants; apply revocation only after cache expiry; widen delegated action scope; log the current rather than decision-time policy version; perform the protected effect before the final decision; and leak denial rationale. Include a reworted Plain policy that is plausible but unsupported and must fail admission rather than borrow the curated profile.

The benchmark author must explicitly resolve denial precedence, decision-time semantics, clock assumptions, cache bounds, delegation depth, audit privacy, and whether revocation affects already-started operations. A human reviewer should be able to locate each choice in the source and resulting contract.

A.4 Benchmark C: temporal ML evaluation and deployment decision

Why this case

This benchmark extends the existing temporal-lineage fixture into a realistic data/estimation task. It should test whether Plain2Metta can preserve dataset identity, temporal splits, feature availability, baselines, uncertainty, selection criteria, and deployment conditions without pretending that symbolic execution itself validates a predictive model.

System envelope to specify

Choose a small public or generated time series with an immutable local snapshot. Define an as-of forecasting or risk-estimation task, prediction horizon, feature timestamps, train/validation/test windows, a named naive baseline, a metric, an uncertainty procedure, and a decision rule. Record which transformations are fit using which window. If model selection occurs, separate selection data from final evaluation data.

Candidate obligations should include: no future or unavailable feature enters training or inference; split timestamps and data hash are reported; all competitors use identical evaluation cases; the named baseline is computed; metric direction and aggregation are correct; uncertainty is estimated by a declared method; missingness and degenerate windows fail explicitly; and a deployment recommendation is conditional on preregistered thresholds rather than narrated after seeing results. Claims about future distribution shift should normally remain assumptions or operational obligations.

Expected representations and evidence

The IR should represent dataset versions, event time, availability time, feature lineage, transforms, windows, horizons, metrics, comparators, uncertainty procedures, seeds, and decision thresholds. MeTTa should expose lineage and eligibility relations and support queries about whether a feature was available as of a forecast origin. Grounded Python should perform the actual numerical fitting and evaluation in an isolated, deterministic local environment.

Hypothesis should generate boundary timestamps, missing values, duplicated records, empty windows, and transformations to test leakage and metric invariants. Z3 may check finite ordering constraints and mutually inconsistent window declarations. Lean can prove a small property of split eligibility or aggregation over a simplified data type. TLC is appropriate only if a stateful streaming/deployment protocol is included; otherwise it should be marked inapplicable. Scientific performance evidence is not automatically G6: operational grade requires a monitored deployed interval and drift/error policy.

Mutants and questions the author must settle

Mutants should include: fit normalization on all data; leak the target through a lag error; reverse the metric comparison; compare models on different rows; silently drop hard cases; select and evaluate on the same window; misalign the horizon; report a confidence interval computed over inappropriate independent units; or change the data while retaining an old digest in the report.

The author must settle event versus ingestion time, timezone and boundary conventions, missing-data policy, stochastic seed policy, resampling unit, multiple-comparison treatment, practical versus statistical significance, and what evidence would justify deployment. The benchmark should reward an honest inconclusive result when the fixture does not support a superiority claim.

A.5 Benchmark D: evidence-based policy decision under conflict and ignorance

Why this case

The first three cases emphasize operational correctness. This case should exercise MeTTa’s prospective role in explicit reasoning with incomplete, conflicting, differently sourced evidence. It tests whether the pipeline can preserve epistemic distinctions instead of collapsing absent evidence, negative evidence, disagreement, and low confidence into one Boolean.

System envelope to specify

Define a bounded decision-support setting such as approving an automated action, escalating it to a human, or declining to decide. Inputs should include claims from sources with declared identity and provenance, positive and negative observations, source dependence, policy constraints, and one or more missing critical facts. Keep the domain non-sensitive and the policy small enough that expected conclusions can be independently enumerated.

Candidate obligations should cover: preserve evidence provenance and polarity; distinguish ignorance from balanced conflict; prevent two dependent reports from counting as independent corroboration; apply explicit policy constraints; expose which premises support a recommendation; and return abstain/Unknown when a critical threshold or fact is absent. The benchmark should include a case where adding evidence retracts or weakens a prior conclusion, exercising non-monotonic update and downstream invalidation.

Expected representations and evidence

The Plain source should define the decision policy without requiring the author to write MeTTa. The IR should represent claims, observations, sources, dependence, timestamps, weights or evidence measures, policy rules, contradictions, and explicit unknowns. MeTTa should own the inspectable reasoning graph and queries that return conclusions with supporting evidence, not merely a final label. Python may supply a grounded evidence aggregation calculation if its mathematical interpretation is specified independently.

Hypothesis should generate evidence-set permutations, duplicates, dependent copies, missing facts, and contradictory packets, checking invariance and abstention properties. Z3 can test consistency of a bounded crisp policy fragment. Lean may prove structural properties such as duplicate-insensitivity under an explicitly defined aggregation rule. TLC is likely inapplicable unless the benchmark adds a workflow in which evidence arrives over time and actions become irreversible. No backend should be allowed to manufacture a ground-truth policy choice that the Plain source leaves unresolved.

Mutants and questions the author must settle

Mutants should: treat missing evidence as false; double-count duplicates; ignore source dependence; discard negative evidence; overwrite rather than retain contradiction; threshold before applying a mandatory policy constraint; lose provenance during aggregation; or convert abstention into the apparent majority class. Expected results should explicitly include ignorance, conflict, supported decision, prohibited decision, and insufficient-policy states.

The author must choose the evidence semantics, dependence representation, revision behavior, threshold ownership, explanation requirements, and the boundary between a mathematical aggregation result and a normative decision. If these choices are still research questions, the benchmark should expose them as variants rather than pick one invisibly.

A.6 Portfolio-level acceptance

The four cases should share artifact schemas and backend interfaces but should not be forced into one universal semantic profile. Portfolio acceptance should require at least four distinct semantic shapes to traverse the general pipeline without exact-source conditional branches in the orchestrator. Each case should reconstruct its ancestry, kill an agreed preregistered set of relevant mutants, preserve justified Unknowns, and render tool scopes and assumptions in the API/UI. Surviving mutants must be reviewed and classified; the score itself is not a probability of correctness.

The benchmark-writing phase is complete only when a reviewer could implement the cases without inventing the missing semantics and could tell, before running the system, which observations would support, refute, or leave each obligation unresolved.

B Two work plans for the next development phase

Both plans begin from the merged prototype and its accepted Stage 1–11 vertical. They are not estimates for rebuilding that infrastructure. They cover authoring the realistic benchmarks, making the minimum generalizations those benchmarks require, integrating the four verticals, and producing reproducible acceptance evidence. Calendar estimates assume local tools and dependencies remain available and no new production deployment, paid compute, or unrestricted model-provider integration enters scope.

B.1 Plan S: one coding agent

The single-agent plan optimizes for semantic continuity and low coordination cost. Its principal risk is a self-confirming implementation: one agent may write the interpretation, lowering, oracle, and test in mutually compatible ways. The plan therefore inserts explicit review and mutation checkpoints and requires the agent to freeze intent artifacts before implementation.

Phase	Deliverable	Work and exit gate	Indicative time
S0	Baseline and interface map	Reproduce current focused/full suites and all three live examples; map extension seams; record exact branch, commit, tools, and known dirty state.	1–2 days
S1	Benchmark design pack	Write the four complete Plain tasks, obligation matrices, backend ownership, fixtures, expected Unknowns, and preregistered mutants. Obtain review before code.	4–6 days
S2	Shared semantic spine	Generalize exact-source admission, profile dispatch, contract-pattern registry, fixture loading, and evidence projection only as required by the frozen designs. Preserve old examples.	3–5 days
S3	Transaction benchmark	Implement the end-to-end reservation/payment slice, focused properties/model, mutants, MeTTa/Python observations, and UI evidence.	4–6 days
S4	Authorization benchmark	Implement the concurrent revocation slice and its independent oracles/mutants.	4–6 days
S5	ML benchmark	Implement temporal lineage, numerical runner, statistical procedure, properties, and honest scientific/operational boundaries.	4–6 days
S6	Epistemic benchmark	Implement evidence/provenance reasoning, conflict/ignorance cases, and abstention-preserving oracles.	4–6 days
S7	Cross-case integration	Remove case-specific core branches, reconcile schemas, run all cases after every relevant source/contract mutation, and review surviving mutants.	3–5 days
S8	Acceptance and report	Clean-checkout full suite, pinned backends, trace reconstruction, UI smoke, hygiene and secret checks, independent audit, and final documentation.	2–4 days

This gives roughly 29–46 focused working days, or six to nine calendar weeks allowing for review and debugging. The sequence is intentionally benchmark by benchmark: the agent should finish a thin vertical before expanding the next, so infrastructure is generalized from observed needs. If a common abstraction emerges during S3, it should be validated against at least the existing examples and the next benchmark design before becoming core architecture.

The agent should maintain one task branch or isolated worktree, commit coherent increments, and keep a running experiment ledger. Each phase begins with a red acceptance test or an immutable design artifact and ends with focused tests, broader regression, diff hygiene, and a project-record update. At S1, S4, S6, and S8, use an independent reviewer—human or separately prompted frontier model with frozen inputs—to challenge semantics rather than merely style.

B.2 Plan P: six parallel coding agents

The six-agent plan exploits the modular seams already built. It does not run the old 11 stages independently six times. It freezes shared contracts, then assigns vertical ownership so most files and semantic decisions are disjoint. One agent owns integration continuously; benchmark agents do not casually edit the shared composer, store, or public API schemas.

Agent	Primary ownership	Responsibilities
P0	Architecture/integration	Freeze shared schemas and interfaces; maintain integration branch; review any core change; run continuous contract/regression tests; resolve cross-case ancestry and API/UI consistency.
P1	Transaction case	Author and implement saga contracts, TLC/Hypothesis schedules, local proofs/checks, reference runtime, mutants, and evidence views within its vertical.
P2	Authorization case	Author and implement identity/policy/revocation semantics, concurrency model, policy checks, mutants, and evidence views.
P3	Temporal ML case	Author data/split/metric/uncertainty contract, deterministic local dataset and runner, leakage properties, mutants, and scientific claim boundaries.
P4	Epistemic policy case	Author evidence/conflict/ignorance semantics, MeTTa reasoning graph, aggregation oracle, abstention properties, mutants, and evidence views.
P5	Adversarial/acceptance	Independently review benchmark obligations; build cross-case mutation and stale-artifact attacks; maintain clean-checkout acceptance, reproducibility records, and later documentation/audit.

Day 0–2: freeze the collaboration contract

P0 publishes a versioned benchmark interface pack: canonical artifact fields, contract-pattern extension interface, backend-result envelope, fixture layout, G-vector rules, UI projection contract, and ownership map. P1–P4 draft their benchmark narratives and obligation matrices in parallel. P5 critiques them, looking especially for self-confirming oracles, missing Unknown states, and claims assigned to inappropriate tools. At the gate, every requirement has a stable ID and expected evidence class; unresolved semantics are explicit; the six agents agree which files are shared and which are vertically owned.

This two-day freeze is the main serial prerequisite. Implementation may begin on private fixtures earlier, but no agent may merge a core interface assumption that has not passed the gate.

Day 3–7: four verticals and the shared spine in parallel

P1–P4 implement the smallest end-to-end slice of their case in independent worktrees. Each slice includes source, contracts, one MeTTa observation, one grounded observation, at least one independent property or formal check, one deliberately Unknown obligation, and two semantic mutants. P0 extracts only the shared generalizations needed to admit those slices and integrates at least daily. P5 begins adversarial tests against each accepted interface and checks that source or contract changes invalidate all appropriate descendants.

Benchmark branches rebase or merge from the integration spine frequently; they do not merge each other. A change to a shared schema requires P0 review, compatibility tests for the old examples, and notification to all owners. Generated fixtures and golden outputs remain case-owned data rather than conditional code in the orchestrator.

Day 6–10: deepen evidence while integrating continuously

Once a thin slice is accepted, P1–P4 add the remaining state shapes, backend tasks, mutants, counterexample presentation, and exact traceability. P5 runs cross-case attacks: stale plan reuse, swapped evidence, incorrect backend attribution, shared-dependency overpromotion, runtime disagreement, timeout, and semantic-preserving reformatting versus meaning-changing edits. P0 removes accidental case coupling, stabilizes API/UI rendering, and runs the full suite on the integration branch.

The gate is not “all agents report done.” It is one integrated commit from which every old and new case traverses the general pipeline, every obligation has a server-derived verdict, all preregistered mutants have a recorded disposition, and justified Unknowns remain visible.

Day 10–12: convergence and independent challenge

Feature work stops except for failures found by acceptance. P5 owns the clean-checkout candidate and reproduces the suite from recorded commands. P0 audits shared code and ancestry. P1 reviews P2’s semantics, P2 reviews P3, P3 reviews P4, and P4 reviews P1, so no benchmark is accepted solely by its author. Reviews compare Plain requirements, contracts, oracles, mutants, and rendered evidence; they do not merely inspect code style.

Surviving mutants are classified as equivalent, out of scope, oracle gaps, or implementation defects with evidence. Any critical oracle gap reopens the owning vertical. Tool timeouts or inapplicability remain explicit results and are not patched into Pass.

Day 12–15: release-candidate evidence and documentation

P5 runs the final clean-checkout acceptance and artifact/hash checks. P0 prepares the integrated diff and rollback/restart notes. Benchmark owners finalize exact case documentation and reproduce one another’s commands. An independent frontier-model or human audit receives frozen source, contracts, results, and known limitations. Findings receive explicit accept/reject/defer decisions. Merge, deployment, or public exposure remains a separate authorization decision.

Under these assumptions the realistic target is 10–15 working days, or about two to three calendar weeks. Six agents do not provide a sixfold speedup: the two-day contract freeze, integration latency, final mutation campaign, and independent audit remain serial. The gain comes from parallel benchmark verticals and continuous integration, not from assigning multiple agents to edit the same core files.

B.3 Coordination controls for the parallel plan

The parallel plan should use the following controls:

- one worktree and task branch per agent, with a declared file-ownership map and no direct pushes to the default branch;
- one versioned interface pack and contract-test suite that all verticals consume;
- P0 approval for shared schemas, canonical serialization, composer, artifact-store, and public API changes;
- daily integration rather than a final “big bang” merge;
- machine-readable fixtures and expected evidence, minimizing merge conflicts in central code;

- focused tests on each branch and the full provider-free/vertical suite on the integration branch;
- preregistered semantic mutants owned or independently reviewed by P5;
- exact commit/hash references in every handoff, so agents cannot validate different versions while believing they share a baseline;
- stop-the-line treatment for provenance, invalidation, evidence attribution, and overpromotion defects; and
- a final cross-review rotation and clean-checkout reproduction before any completion claim.

If only three or four agents are available, retain P0 integration and P5 adversarial independence, then combine benchmark ownership by semantic affinity. If more than six are available, add reviewers, fixture/oracle authors, and formal-backend specialists rather than splitting the shared core among more simultaneous editors. Coordination quality, not raw agent count, sets the useful parallelism ceiling.